Guia Rápido do Jupyter Notebook

O Jupyter Notebook é um interfarce gráfica que permite a edição de notebooks em um navegador web, tais como Google Chrome ou Firefox.
O nome Jupyter é um acrônimo criado a partir das linguagens de programação que inicialmente foram aceitas pelo Projeto Jupyter: Julia, Python e R.
Além dessas, hoje, o Projeto Jupyter suporta também C++, Ruby, Fortran e outras.

Forma de Operação

Abrindo o Jupyter

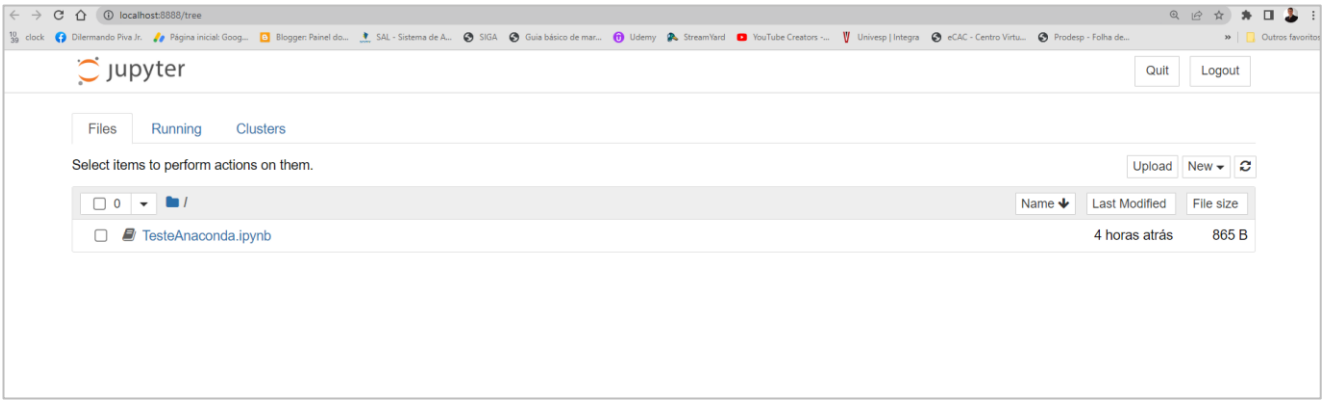
Depois de instalado, você pode abrir o Jupyter Notebook pelo terminal (Prompt de Comandos), digitando o seguinte comando: **jupyter notebook**.
É aconselhável que, antes de digitar o comando acima, você já esteja no diretório (ou pasta) onde deseja guardar seus notebooks (arquivos).

```
Prompt de Comando
Microsoft Windows [versão 10.0.22621.1105]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\pivaj>cd C:\CienciaDados\Modulo01

C:\CienciaDados\Modulo01>jupyter notebook
```

Será então inicializado o servidor do Jupyter Notebook, ficando o terminal, mostrando que o servidor está em execução, conforme a tela baixo.
ATENÇÃO: Não feche o terminal. É preciso que o servidor permaneça em execução para que você possa realizar a edição e execução de notebooks.

```
Administrador: Prompt de co
[I 14:35:42.755 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 14:35:42.783 NotebookApp]

    To access the notebook, open this file in a browser:
        file:///C:/Users/pivaj/AppData/Roaming/jupyter/runtime/nbserver-16736-open.html
    Or copy and paste one of these URLs:
        http://localhost:8888/?token=33f82bfa7989cbb0d53d5a70b6e4336ff856a9710d9d886a
     or http://127.0.0.1:8888/?token=33f82bfa7989cbb0d53d5a70b6e4336ff856a9710d9d886a
[I 14:36:27.794 NotebookApp] Interrupted...
[I 14:36:27.794 NotebookApp] Shutting down 0 kernels
[I 14:36:27.794 NotebookApp] Shutting down 0 terminals

c:\CienciaDados\Modulo01>
c:\CienciaDados\Modulo01>jupyter notebook
[I 2023-02-02 14:36:40.010 LabApp] JupyterLab extension loaded from C:\Users\pivaj\anaconda3\lib\site-packages\jupyterla
b
[I 2023-02-02 14:36:40.010 LabApp] JupyterLab application directory is C:\Users\pivaj\anaconda3\share\jupyter\lab
[I 14:36:40.015 NotebookApp] Serving notebooks from local directory: c:\CienciaDados\Modulo01
[I 14:36:40.015 NotebookApp] Jupyter Notebook 6.4.12 is running at:
[I 14:36:40.015 NotebookApp] http://localhost:8888/?token=06d96b78299b9717427f84955d3c67771b65bd387d7f34e3
[I 14:36:40.015 NotebookApp]  or http://127.0.0.1:8888/?token=06d96b78299b9717427f84955d3c67771b65bd387d7f34e3
[I 14:36:40.015 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 14:36:40.038 NotebookApp]

    To access the notebook, open this file in a browser:
        file:///C:/Users/pivaj/AppData/Roaming/jupyter/runtime/nbserver-8172-open.html
    Or copy and paste one of these URLs:
        http://localhost:8888/?token=06d96b78299b9717427f84955d3c67771b65bd387d7f34e3
     or http://127.0.0.1:8888/?token=06d96b78299b9717427f84955d3c67771b65bd387d7f34e3
```

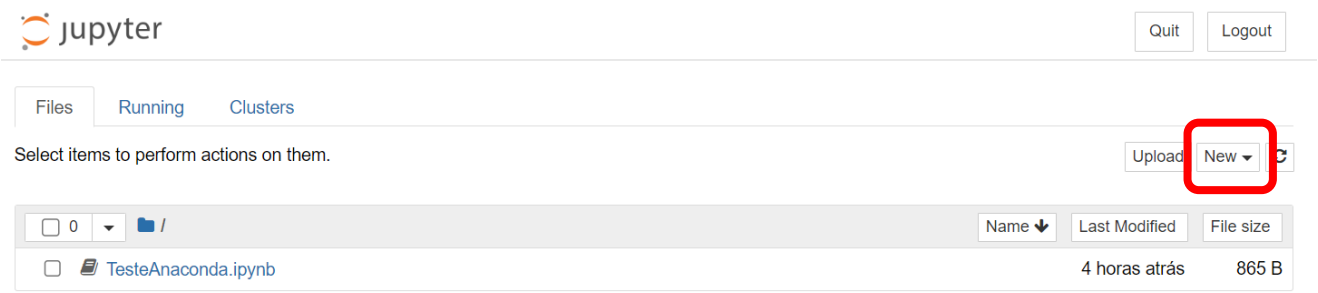
Uma vez que tudo esteja funcionando adequadamente, será aberto em seu Browser Padrão a janela principal do Jupyter Notebook, conforme mostrado na figura a seguir.



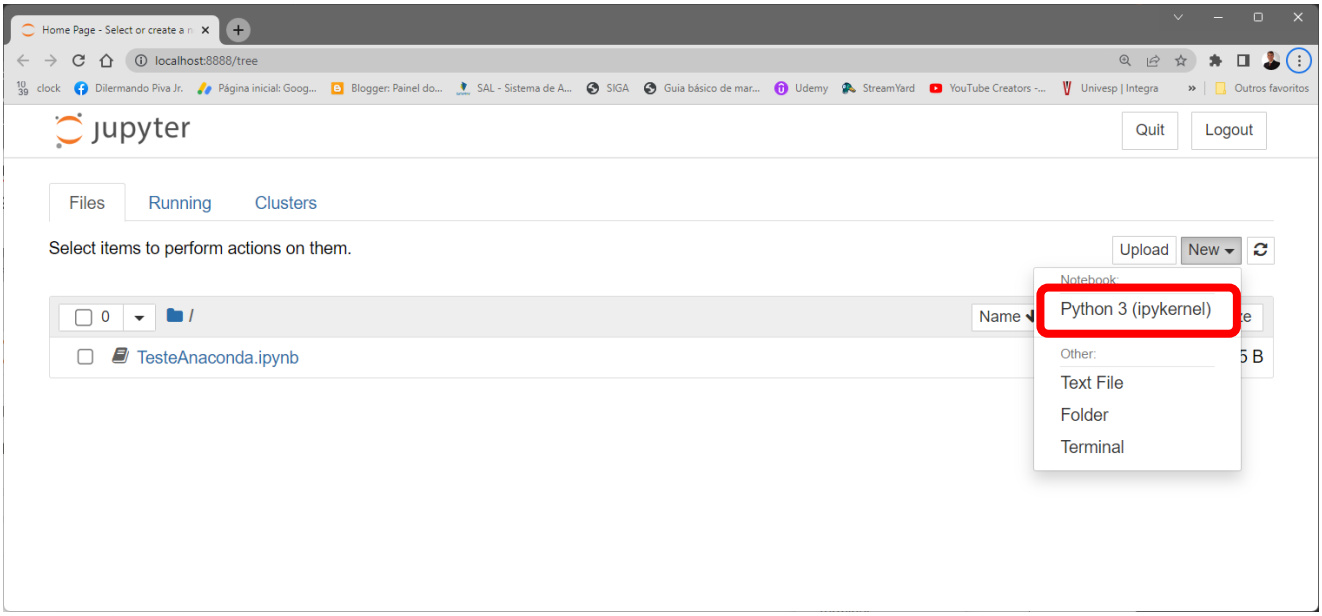
Com essa página, além de servir como um navegador simples de diretórios, você pode abrir notebooks já existentes, criar novos notebooks, abrir terminais do sistema, criar e deletar diretórios, e criar e editar arquivos de texto.

Abrindo ou editando um notebook

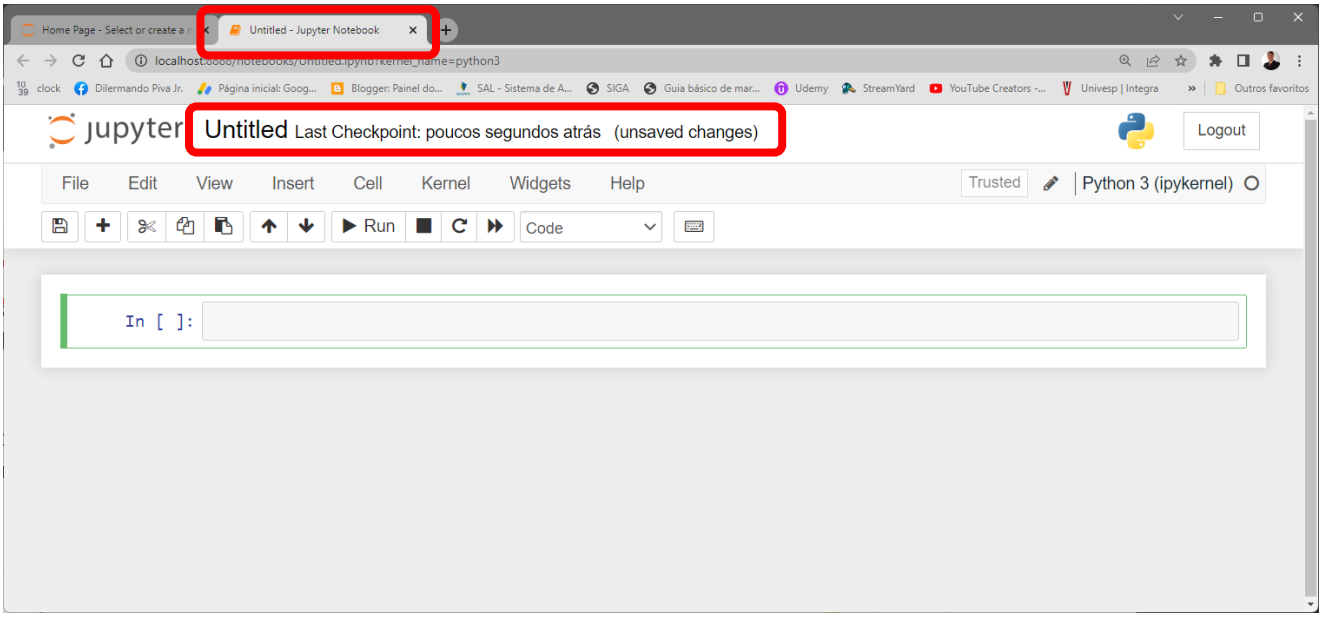
Para criar um notebook selecione o botão *new* no canto superior esquerdo da página:



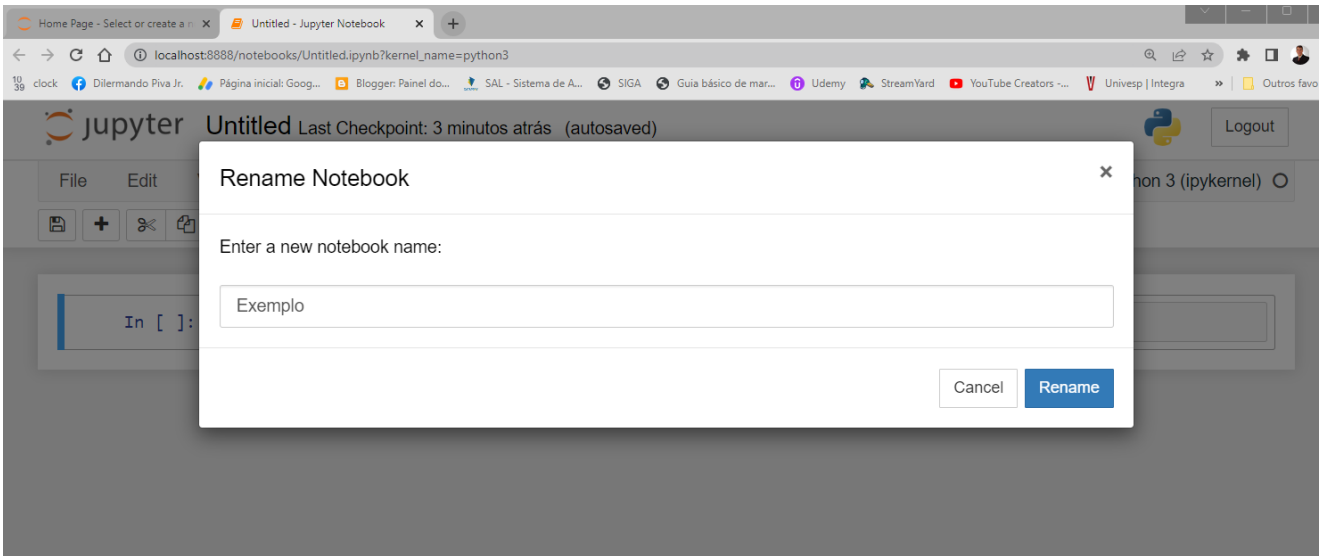
O Jupyter dará a opção de você criar um notebook cuja execução será suportada pelo *kernel* de uma linguagem, no caso da figura abaixo o Python 3:



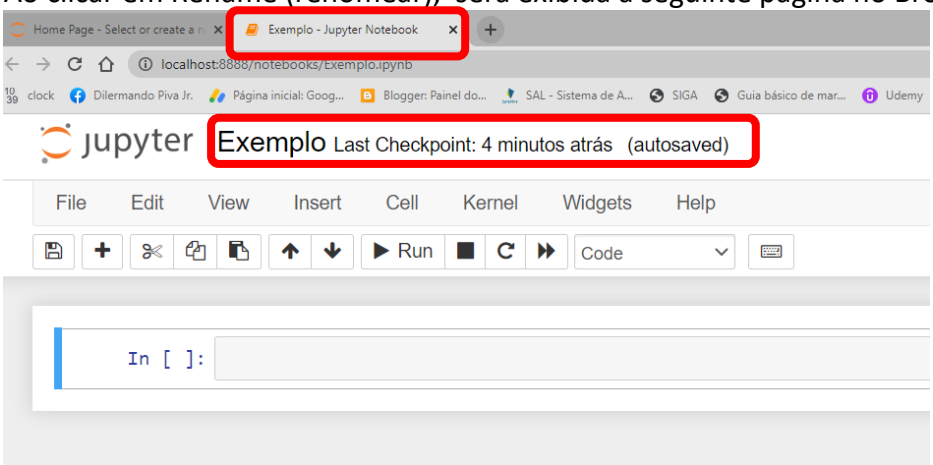
Note que ele vai criar uma nova aba no navegador com o nome: Untitled



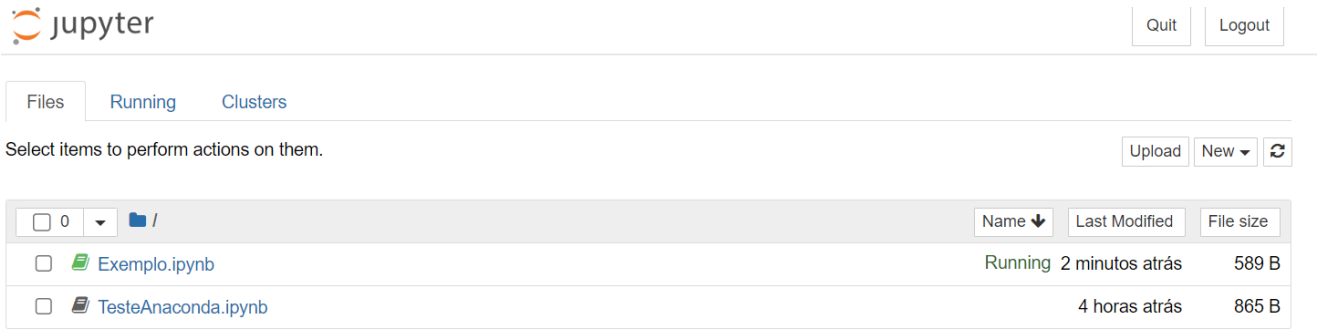
Para alterar o nome, clique sobre "Untitled" ao lado do logo do Jupyter. Entrará em modo de edição. Basta colocar o novo nome, no caso, vamos alterar para: Exemplo



Ao clicar em Rename (renomear), será exibida a seguinte página no Browser:

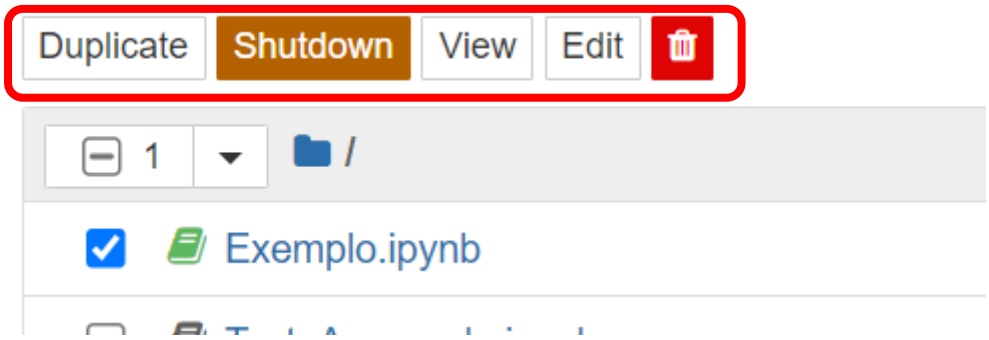


Clique no botão salvar ou em “File” → “Save and Checkpoint” ou tecle: Ctrl + S para salvar o arquivo.
Em seguida feche a guia atual. Você voltará para a tela inicial, conforme ilustrado na página web a seguir:

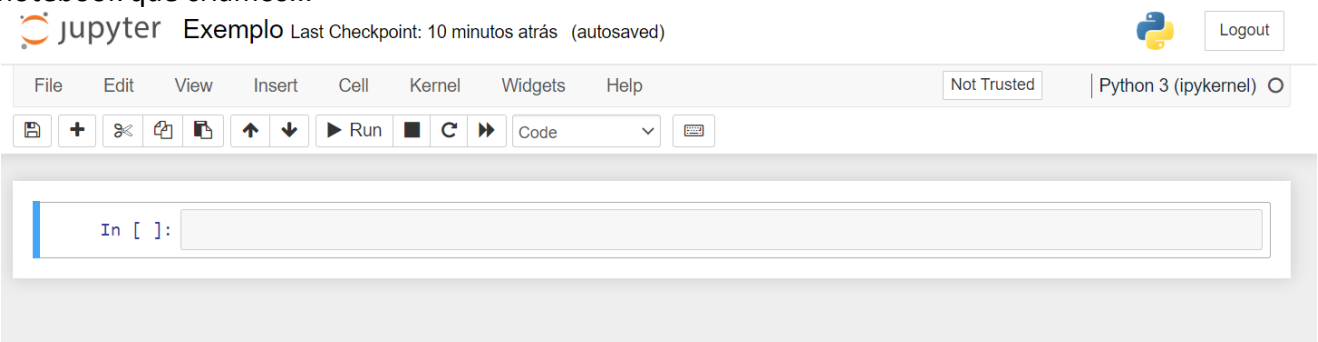


E aí está o “notebook” que acabamos de criar: **Exemplo.ipynb**

Para abrir um notebook já existente, basta procurar por um arquivo do tipo .ipynb (de Ipython Notebook) com o auxílio da pasta inicial do Jupyter. Aí você pode clicar na caixa de seleção a frente do nome do arquivo desejado e escolher uma das ações que ele exibir ... conforme mostra a figura abaixo:



Vamos clicar em View (Visualizar) e começar a programar e documentar o trabalho neste novo notebook que criamos...



Modos de uma célula no Jupyter Notebook

Uma célula do Jupyter pode assumir dois estados: modo de edição ou o modo de comando.
No modo de edição, o usuário irá apenas editar o conteúdo da célula, seja esse um código ou um texto *markdown*, por exemplo.
Já o modo comando permite o usuário manipular a célula: executar o código ou compilar o texto *markdown*, deletar a célula, copiar a célula, recortar a célula, criar uma nova célula, mudar a célula para aceitar código ou para aceitar *markdown*, fundir duas ou mais células entre outras coisas.

A célula quando estiver em modo de edição terá uma margem verde:

```
In [ ]: x = 1
```

Ao passo que uma célula em modo de comando terá margem azul:

```
In [ ]: x = 1
```

Tipos de células

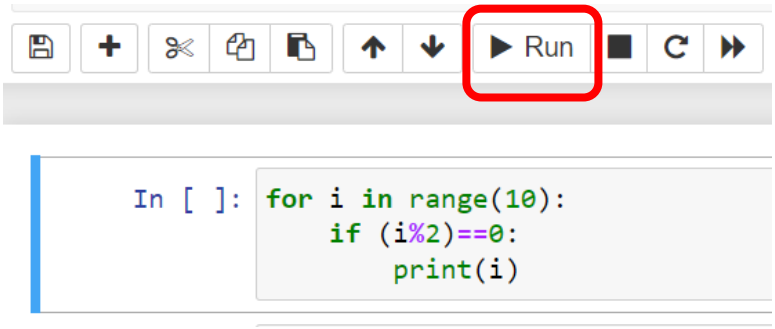
Existem três tipos de células no Jupyter: **código**, ***markdown*** e ***raw NBconvert***.

As células de *código* são aquelas que permitem a execução de códigos de uma determinada linguagem.
As células do tipo *markdown* permitem a utilização da linguagem de marcação *Markdown*, que permite a edição de textos.
Já as *raw N Bconvert* permitem a execução de códigos externamente, tais como o LaTeX ou o sistema operacional.

A célula de código sempre se apresentará da seguinte forma:

```
In [ ]: for i in range(10):
            if (i%2)==0:
                print(i)
```

Uma vez selecionada uma determinada célula de código, podemos executá-lo (run). Apenas a célula selecionada.
Basta, com a célula selecionada, clicar no botão “run”



Ao fazê-lo, será mostrado logo abaixo da célula a saída (output) do código. No caso em específico, serão mostrados todos os números pares inferiores a 10.

```
In [1]: for i in range(10):
            if (i%2)==0:
                print(i)

        0
        2
        4
        6
        8
```

saída

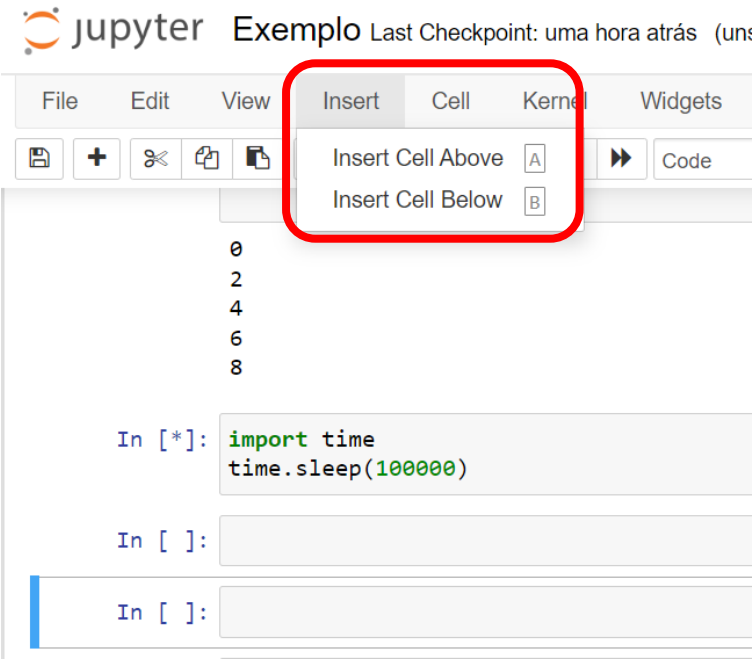
Note que dentro do colchete foi colocado o número [1]. Esse número representa a ordem de execução das células.

Enquanto a célula estiver "rodando", um asterisco irá aparecer no seu canto direito para indicar que a célula ainda está executando o código:

```
In [*]: import time
        time.sleep(100000)
```

Criando novas células

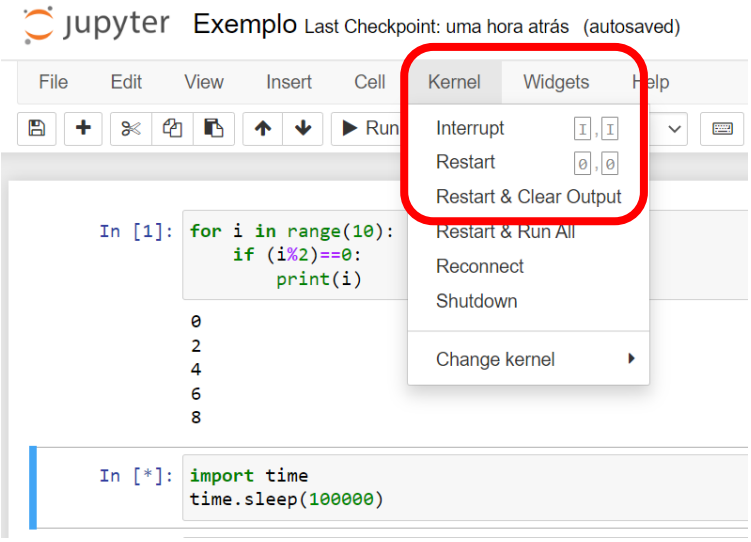
Dada qualquer célula no modo de comando, pode-se acrescentar uma nova célula acima ou abaixo da mesma.
Para tanto você pode clicar no botão cell (célula) → insert (inserir) e escolhendo above (acima) ou bellow (abaixo).
Estando no modo de comando você também pode teclar o atalho: tecla "a" para inserir acima e a tecla "b" para inserir abaixo.



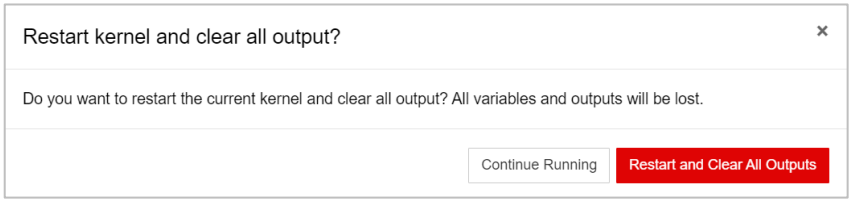
ATENÇÃO:
Note que a frente das opções "Insert Cell Above" e "Insert Cell Bellow" existe um retângulo com uma letra... Essa é a tecla de atalho (estando a célula em modo de comando)

Parando a execução ou interrompendo o kernel

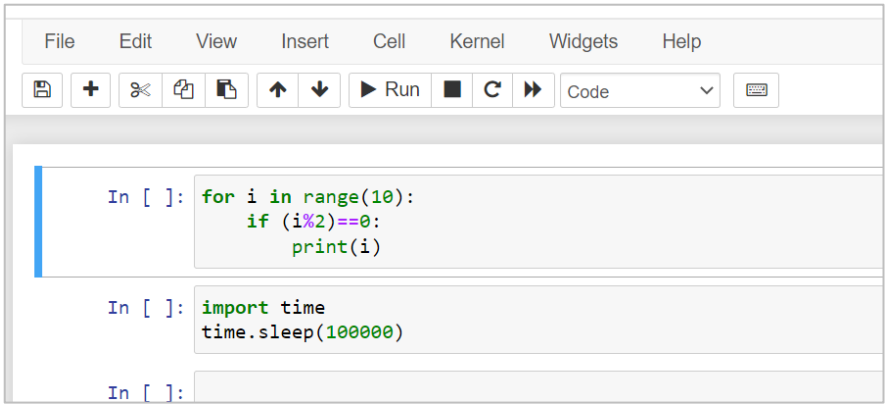
Para interromper, restaurar e restaurar e limpar a saída, utilizamos as opções ilustradas na imagem a seguir.



Ao escolher a opção “Restart and Clean Output” (restaurar e limpar a saída), será solicitada a confirmação...



Após confirmar, teremos o seguinte resultado:



Atalhos do teclado

Os seguintes atalhos estão implementados por default para célula no modo comando:
Ctrl-Enter. executa a célula;
Shift-Enter. executa a célula e cria uma nova célula abaixo no modo de comando;
Alt-Enter. executa a célula e cria uma nova célula abaixo no modo de edição;
Y muda a célula para o tipo código;
M. muda a célula para o tipo markdown;
A: insere célula acima;
B: insere célula abaixo;
D + D: deleta uma célula
X: recorta a célula;
C. copia a célula;
Z: desfaz o apagar célula; e
L: ativa a enumeração das linhas.

Os seguintes atalhos estão implementados por default para célula no modo de edição:
Tab: autocompleta durante a digitação; e
Shift-Tab: abre uma mini-janela com parte da documentação sobre a parte do código durante a documentação. A auto-completação durante a digitação permite também que o usuário escolha o que deve ser completado caso haja mais de uma opção:
O **shift-Tab** é muito útil quando se esquece, por exemplo, os argumentos de uma função.

Execução

O Jupyter Notebook tem diversos comandos próprios que podem ser acessados **%** no início de uma célula. Por exemplo, **%pwd** mostra o endereço do diretório no qual o notebook se encontra.

```
In [1]: %pwd

Out[1]: 'c:\\CienciaDados\\Modulo01'
```

Tempo de execução

O comando **%%time** exibe o tempo de execução da célula em que o comando foi executado.

```
In [4]: %%time
        a = 0
        for i in range(100000):
            for j in range(1000):
                a = i + j

        Wall time: 8.72 s
```

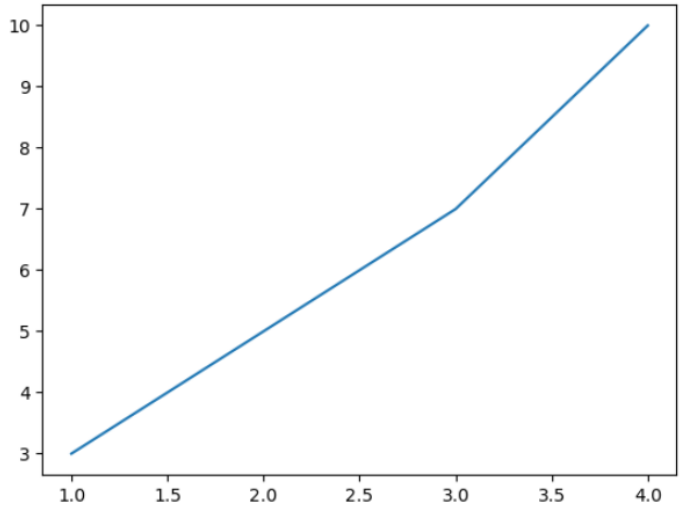
Matplotlib

Diferente do que é feito na linha de comando, o Jupyter Notebook não precisa do **plt.show()** para mostrar a figura. Ao executar **%matplotlib inline** em uma das células (só é necessário executá-lo uma vez), a figura é mostrada na própria página, no output.
Esse comando é necessário, pois do contrário o Jupyter ira exibir apenas a representação do objeto:

```
In [5]: import matplotlib.pyplot as plt
        plt.plot([1,2,3,4],[3,5,7,10])

Out[5]: [<matplotlib.lines.Line2D at 0x26181814370>]
```

Após executar o **%matplotlib inline**, o gráfico é exibido abaixo do output:

```
In [1]: %matplotlib inline

In [2]: import matplotlib.pyplot as plt
        plt.plot([1,2,3,4],[3,5,7,10])

Out[2]: [<matplotlib.lines.Line2D at 0x21665390c10>]
```



A partir da versão xx, a opção “inline” é padrão.

O comando **%matplotlib notebook** permite a criação de uma gráfico interativo, parecido com a janela nativa do Matplotlib. Porém, isso deixa o programa mais lento.

Interagindo com o sistema operacional

Podemos invocar comandos do sistema operacional sem sair do Jupyter. Por exemplo, **dir** é um comando do Windows que lista o conteúdo de um diretório. Assim, podemos invocá-lo com no Jupyter executando o comando **!dir** em alguma célula:

```
In [3]: !dir

 O volume na unidade C nÆo tem nome.
 O N£mero de S,rie do Volume , 1468-BC95

 Pasta de c:\CienciaDados\Modulo01

02/02/2023  19:39    <DIR>          .
02/02/2023  12:10    <DIR>          ..
02/02/2023  17:04    <DIR>          .ipynb_checkpoints
02/02/2023  19:39            22.314 Exemplo.ipynb
02/02/2023  12:40               865 TesteAnaconda.ipynb
               2 arquivo(s)         23.179 bytes
               3 pasta(s)   1.827.529.195.520 bytes dispon¡veis
```

Uma aplicação muito útil é executar o pip direto no Jupyter:

! pip install xxxx